Q1. The "Clean" Filter

Python

```
from typing import List

def get_positive_evens(numbers: List[int]) -> List[int]:
    """Returns a list of positive even integers using a list comprehension."""
    return [num for num in numbers if num > 0 and num % 2 == 0]
```

Q2. Data Normalizer

Python

```
from typing import List, Dict

def normalize_sensor_data(data: List[Dict]) -> Dict[int, float]:
    """Filters invalid readings and maps IDs to values."""
    return {item["id"]: item["val"] for item in data if item["val"] >= 0}
```

Q3. File Processor (Walrus)

Python

```
def process_file_stream(f):
    """Processes lines using the walrus operator until EOF or SHUTDOWN."""
    while (line := f.readline().strip()):
        if "SHUTDOWN" in line:
            print("Shutdown signal received. Stopping.")
            break
        print(f"Processing: {line}")
```

Q4. Argument Collector

Python

```
def log_event(event_name: str, *args, **kwargs) -> None:
    """Prints event name, tags from args, and metadata from kwargs."""
    print(f"Event: {event_name}")

    if args:
        print("Tags:", ", ".join(args))

    if kwargs:
        print("Metadata:")
        for key, value in kwargs.items():
            print(f"  {key}: {value}")

# Example: log_event("SYS_START", "prod", "web", version=1.2, status="ok")
```

Q5. Memory-Efficient Iterator

Python

```
from typing import Generator

def count_up_to(n: int) -> Generator[int, None, None]:
    """Generator function that yields numbers up to n."""
    for i in range(1, n + 1):
        yield i

# Iterating over the generator
for number in count_up_to(5):
    print(f"Count: {number}")
```

ASSESSMENT QUESTIONS

Q1. Method Manipulation & List Processing

Python

```
from typing import Dict, List


def parse_logs(logs: List[str]) -> Dict[str, str]:
    """Strips, uppercases, and splits log lines into a level-to-message dictionary."""
    log_dict = {}
    for log in logs:
        cleaned = log.strip().upper()
        if ":" in cleaned:
            # Split only on the first colon occurrence
            level, message = cleaned.split(":", 1)
            log_dict[level.strip()] = message.strip()
    return log_dict


# Example usage:
# logs_input = ["  error: disk full  ", "  info: system ok  "]
# print(parse_logs(logs_input))  # Output: {'ERROR': 'DISK FULL', 'INFO': 'SYSTEM OK'}
```

Q2. Conditional Iteration & Accumulation

Python

```
from typing import List, Dict, Union


def get_average_reading(readings: List[Dict[str, Union[str, float]]], target_sensor: str) -> float:
    """Calculates the average value for a specific sensor type. Returns 0 if none found."""
    total = 0.0
    count = 0

    for item in readings:
        if item.get("sensor") == target_sensor:
            total += float(item.get("val", 0))
```

```
            count += 1

    return total / count if count > 0 else 0.0

# Example usage:
# readings_input = [{"sensor": "temp", "val": 25}, {"sensor": "pressure", "val": 10}, {"sensor": "temp", "val": 30}]
# print(get_average_reading(readings_input, "temp"))  # Output: 27.5
```

Q3. Advanced Function Argument Patterns

Python

```
def configure_pipeline(*args: str, **kwargs: Union[str, int, float]) -> str:
    """Formats positional stages and keyword settings using .join() and list comprehensions."""
    stages_str = ", ".join(args)
    settings_str = ", ".join([f"{k}={v}" for k, v in kwargs.items()])

    return f"Stages: [{stages_str}] | Settings: [{settings_str}]"

# Example usage:
# print(configure_pipeline("extract", "transform", "load", retries=3, timeout=30))
# Output: Stages: [extract, transform, load] | Settings: [retries=3, timeout=30]
```

Q4. Logic with break/else

Python

```
from typing import List

def find_first_error(log_entries: List[str]) -> str:
    """Returns the first log entry containing an error, or 'NO_ERRORS_FOUND' via for-else."""
    for entry in log_entries:
        if "ERROR" in entry.upper():
            return entry
    else:
        return "NO_ERRORS_FOUND"
```

```
# Example usage:
# print(find_first_error(["System starting...", "Warning: high memory", "ERROR: crash"]))
# Output: ERROR: crash
# print(find_first_error(["All systems normal"]))
# Output: NO_ERRORS_FOUND
```

Q5. Functional Transformation

Python

```
from typing import List, Union

def process_data(data_list: List[Union[int, float, str]]) -> List[float]:
    """Cleans mixed data types and filters values greater than 25.0."""
    result = []

    for item in data_list:
        # Convert strings to floats after stripping, or cast numbers directly
        if isinstance(item, str):
            value = float(item.strip())
        else:
            value = float(item)

        if value > 25.0:
            result.append(value)

    return result

# Example usage:
# print(process_data([10, " 20 ", 30.5, " 40.2 "]))  # Output: [30.5, 40.2]
```

Easy (Q1 – Q2)

Q1 Output:

Plaintext

Engineering

Python

- **Why:** Local variable shadowing. Inside func(), defining x = "Engineering" creates a local variable that shadows the global x ("Python") during execution. The global scope remains untouched.

Q2 Output:

Plaintext

4

- **Why:** The .append() method inserts its argument as a single element at the end of the list, resulting in [1, 2, 3, [4, 5]], which has a length of 4.

Intermediate (Q3 – Q8)

Q3 Output:

Plaintext

[1]

[1, 2]

- **Why:** The **Mutable Default Argument Trap**. Default arguments are evaluated once when the function is defined. The list [] is shared across every subsequent call, causing the second call to inherit the leftover item from the first.

Q4 Output:

Plaintext

30

- **Why:** The nonlocal x keyword tells Python to look in the nearest enclosing scope (outer()), bypassing both local and global scopes. It modifies x from 20 to 30.

Q5 Output:

Plaintext

[20, 40, 60, 80]

- **Why:** Generators are **lazy-evaluated**. The expression (x * 2 for x in vals) is not evaluated until iteration happens (list(gen)). Because vals.append(40) occurred before iteration, the generator processes all four elements (10, 20, 30, 40).

Q6 Output:

Plaintext

(1, (2, 3), {'b': 4, 'c': 5})

- **Why:** Argument unpacking. a captures positional argument 1, *args packs extra positional arguments into a tuple (2, 3), and **kwargs packs keyword arguments into a dictionary {'b': 4, 'c': 5}.

Q7 Output:

Plaintext

Traceback (most recent call last):

File "...", line 6, in <module>

toggle()

File "...", line 4, in toggle

if status == "READY":

UnboundLocalError: local variable 'status' referenced before assignment

- **Why:** Because status = "BUSY" is assigned inside toggle(), Python treats status as a *local* variable for the entire function scope. When it evaluates if status == "READY":, Python checks the local variable before it has been assigned a value, raising an UnboundLocalError.

Q8 Output:

Plaintext

[1, 2, 0]

- **Why:** Dictionary .get(key, default) safely retrieves values. Keys "a" and "b" return 1 and 2, while key "c" is missing, triggering the fallback default value of 0.

Hard (Q9 – Q10)

Q9 Output:

Plaintext

[10, 11, 12]

- **Why:** The factory function factory(i) binds the loop variable i **immediately** as a function argument when called inside the list comprehension, capturing 0, 1, and 2 correctly for each lambda.

Q10 Output:

Plaintext

[1, 2, 3, 4]

[1, 2, 3, 4]

Easy

- **Q1. Answer: C) 8**
    - *Explanation:* Python uses the LEGB rule (Local, Enclosing, Global, Built-in) to resolve variables. Since x is not defined locally inside add_val, Python looks in the global scope, finds x = 5, and evaluates 3 + 5 = 8.
- **Q2. Answer: B) strip()**
    - *Explanation:* .strip() removes leading and trailing whitespace characters. (.lstrip() and .rstrip() handle left and right sides respectively).

Intermediate

- **Q3. Answer: B) The default list is evaluated only once when the function is defined, causing state to persist and leak across subsequent calls.**
    - *Explanation:* This is the classic **mutable default argument trap**. Default values are bound at definition time, meaning every call sharing that default shares the exact same list instance in memory.
- **Q4. Answer: B) Python treats the variable as *local* for the entire function scope upon assignment, causing it to check the local variable *before* the assignment line runs.**
    - *Explanation:* When Python compiles a function body and sees an assignment (x = ...), it flags x as local. If you reference x on the right side of an operator *before* that assignment line, Python looks locally, finds an uninitialized variable, and raises an UnboundLocalError.
- **Q5. Answer: C) The generator yields items one by one lazily, consuming minimal constant memory regardless of the range size.**
    - *Explanation:* List comprehensions allocate memory for all items at once, whereas generator expressions evaluate and yield items on-demand using iterator protocols, keeping memory overhead close to zero ($O(1)$).
- **Q6. Answer: B) args is a tuple, kwargs is a dictionary (dict)**
    - *Explanation:* *args packs extra positional arguments into a tuple, while **kwargs packs keyword arguments into a standard dictionary.
- **Q7. Answer: C) 3**
    - *Explanation:* The .get(key, default) method checks if the key exists. Since "retries" is missing from config, it gracefully falls back to the specified default value of 3.
- **Q8. Answer: B) (1, "sensor_a"), (2, "sensor_b")**
    - *Explanation:* enumerate() wraps an iterable in an indexing wrapper, returning tuples of (index, value). The start=1 argument offsets the counter to begin at 1 instead of 0.

Hard

- **Q9. Answer: B) [12, 12, 12]**

- *Explanation:* This demonstrates **late binding in closures**. The variable i is looked up dynamically when the lambda is *called*, not when it is *created*. By the time the lambdas are executed, the loop has finished and i has settled on its final value of 2, resulting in 10 + 2 = 12 for all elements.

- **Q10. Answer: A) Because += on lists calls the __iadd__ dunder method, which performs an in-place mutation of the existing list object in memory rather than creating a new list.**
    - *Explanation:* Unlike immutable types (like integers or strings) where x += 1 rebinds a new object, mutable objects like lists implement in-place extension via extend() behind the += operator, modifying the underlying object referenced by both variables.